

Enunciado-Final-PLP-16-12-24

1. Se da una estructura Dato a $b = C1 \mid C2 \mid C3$ b (Dato a b) (Dato a b). Definir recursión primitiva, recursión estructural en base a la recursión primitiva, y después una función Split que tome un Dato a b y devuelva dos listas, una con los a y otra con los b, usando el fold o rec.

2. Se toma una extensión del cálculo lambda λ^* . Se le agrega $\emptyset \mid - M : \sigma$. El operador $\rightarrow^*$ indica la clausura transitivo-reflexiva de la reducción. Verdadero o falso:

a. Si λ^* cumple progreso, entonces $M \rightarrow V$ (falso porque puede que no se reduzca un término a valor en un sólo paso)

b. Si λ^* cumple progreso, entonces $M \rightarrow^* V$ (creo que falso porque necesitas terminación también para que se cumpla, aunque con terminación sería verdadero)

c. Si λ^* cumple preservación y progreso, entonces $M \rightarrow^* V$ (creo que sería el mismo razonamiento que el anterior, preservación no debería cambiar nada)

3. Te daba la regla de inferencia para la aplicación. Después daba dos variantes y pedía explicar por qué no funcionaban. Una quitaba la condición de que los contextos de tipado de los términos tenían que unificar para todas las variables compartidas, el otro en vez de generar una incógnita fresca le asignaba una variable de tipo directamente a la aplicación. La idea era dar una inferencia de tipo errónea con cada regla.

4. Se daba el siguiente programa en Prolog

p(a).

p(b).

q(a).

q(c).

r(X,Y) :- q(X), !, p(Y).

s(X,Y) :- p(X), not(r(X,Y)).

s(Y,Y) :- q(c), r(c,Y).

Tenías que dar el árbol de búsqueda para la consulta S(X,Y).

Después te daba dos clausulas, $\{P(X,X)\}$ y $\{\text{not}(P(X, f(Y)))\}$. Tenías que explicar si se podía aplicar la resolvente o no, y de poder hacerlo cuál era el MGU (trivialmente se puede, y queda $S = \{X = f(Y)\}$).

5. Tomando una representación de grafos como lista de aristas en Prolog (ejemplo: [(a,b), (b,c), (d,c)]). Se pide definir los siguientes predicados:

caminosSimples(+G, +S, +E, -P), que pide devolver todos los caminos simples entre S y E. G

se puede asumir acíclico.

aciclico(+G), que devuelve si G es aciclico o no.

caminos(+G, +S, +E, -P), que devuelve todos los caminos entre S y E, para un grafo que puede tener cíclicos.

Finalmente había que analizar la reversibilidad sobre G y S para este último.

6. Probar con deducción natural

$$\forall X(P(X) \vee Q(X)) \rightarrow \neg(\exists Z(\neg P(Z) \wedge \neg Q(Z)))$$

Después daba una fórmula en cálculo lambda que era una función sobre unión disjunta (o sea, un término de tipo $\sigma + \tau$) y pedía dar la fórmula lógica que correspondería a dicho término y justificar por qué correspondía (correspondencia Curry Howard).

7. Se daban las siguientes definiciones en Smalltalk:

Object subclass: #A { ... n: ^2 }

A subclass: #B { ... n: ^super n }

B subclass: #C { ... m: ^1 } C subclass: #D { ...

}

Se pedía mostrar todo el pasaje de mensajes, y el resultado, para la evaluación de (D new) n. Después se pedía el resultado de (D new) m, suponiendo que se hubiera sobrescrito el new de C para que devuelva (B new).

Obtenido de «<https://www.cubawiki.com.ar/index.php?title=Enunciado-Final-PLP-16-12-24&oldid=17741>»

Enunciado-Final-PLP-18-02-25

Sumario

- 1 Haskell
- 2 Induccion Estructural
- 3 Deduccion Natural
- 4 Prolog

Haskell

```
data Form = Prop String
          | And Form Form
          | Or Form Form
          | Neg Form
```

(A) Definir y dar el tipo de `foldForm`

(B) Definir `fnn :: Form -> Bool -> Form` usando `foldForm`. Que pasa una formula `x` a forma normal negada si el booleano es `True` y pasa a la negacion de `x` a forma normal negada si el booleano es `False`.

Ej:

```
fnn (And (Prop "x") Neg(Or (Prop "y") Neg(Prop "z")))) True = (And (Prop "x") (And Neg(Prop "y")
(Prop "z")))
```

```
fnn (And (Prop "x") Neg(Or (Prop "y") Neg(Prop "z")))) False = (Or Neg(Prop "x") (Or (Prop "y")
Neg(Prop "z")))
```

Induccion Estructural

Dados

```
{A0} alt f g [] = []
{A1} alt f g (x:xs) = f x : alt g f xs
```

Demostrar `alt g1 g2 . alt f1 f2 = alt (g1 . f1) (g2 . f2)`

Deduccion Natural

Dadas las nuevas reglas :

$$\frac{\Gamma, \Delta, \neg \tau \vdash \tau}{\Gamma \vdash \Delta \tau} \quad \Delta i$$

$$\frac{\Gamma \vdash \Delta \tau \quad \Gamma \vdash \Delta \neg \tau}{\Gamma \vdash \tau} \Delta_e$$

Demostrar las siguientes formulas:

- a) $\vdash \tau \Rightarrow \Delta \tau$
- b) $\Delta \tau \vdash \Delta \neg \neg \tau$ (ojo, no se puede que $\neg \neg \neg \tau = \neg \tau$)
- c) $\Delta \tau, \Delta \neg \tau \vdash \sigma$ sugiere usar $\perp_e$ y el ítem anterior.

Prolog

En prolog definir λ -terminos como

- `var(X)` donde X es un numero natural. Representa al uso de la variable numero X.
- `lam(X, M)` donde X es un numero natural y M es un λ -termino. Representa la ligadura de la variable numero X con respecto al λ -termino M.
- `app(M, N)` donde M y N son λ -terminos. Representa a la aplicacion.

Ej:

`lam(1, lam(2, lam(3, app(var(1), app(var(2), var(3)))))` es equivalente al termino $\lambda f. \lambda g. \lambda x. f(g\ x)$

(A) Definir `variablesLibres(+M, -L)` que instancia en una lista L las variables libres del termino M.

Ej: `variablesLibres(lam(1, app(var(1), var(2))), L)` instancia `L = [2]`.

(B) Definir `tamano(+M, -T)` que calcula el tamaño de un termino. `var(X)` suma 1. `lam(X, M)` suma `1 + tamano(M)`. `app(M, N)` suma `1 + tamano(M) + tamano(N)`.

Ej: `tamano(app(var(1), lam(1, var(2))))` instancia `T = 4`.

(C) Definir `generarLambdaTerminos(+xs, -M)` que dada una lista de numeros naturales XS instancia en M λ -terminos infinitos. Sugerencia: Crear los λ -terminos en base al tamaño.

Obtenido de «<https://www.cubawiki.com.ar/index.php?title=Enunciado-Final-PLP-18-02-25&oldid=17738>»

Un rato para pensarlo y después lo charlamos en el P.L. de la mañana
Rendir solo pero creo que sirve para practicar Suerte!
Paradigmas de Programación (PLP) - Final 3/7/2026

Ejercicio 1 - Programación funcional

Se define el siguiente tipo algebraico para representar heaps binarios:

```
data Heap a = Vacio | Nodo Int a (Heap a) (Heap a)
  deriving (Show, Eq)
```

En cada nodo, el primer valor de tipo `Int` representa la **prioridad** del elemento almacenado, y el **segundo valor** representa el dato asociado. Diremos que un valor de tipo `Heap a` satisface el **invariante de min-heap** si, para cada nodo, su prioridad es menor o igual que la prioridad de cada uno de sus hijos no vacíos.

Por ejemplo, el siguiente heap satisface el invariante:

```
h1 :: Heap Char
h1 =
  Nodo 1 'a'
    (Nodo 3 'b' Vacio Vacio)
    (Nodo 5 'c'
      (Nodo 8 'd' Vacio Vacio)
      Vacio)
```

En cambio, el siguiente heap no lo satisface, porque el nodo con prioridad 4 tiene un hijo con prioridad 2:

```
h2 :: Heap Char
h2 =
  Nodo 4 'a'
    (Nodo 2 'b' Vacio Vacio)
    (Nodo 7 'c' Vacio Vacio)
```

No se exige ninguna propiedad de balanceo ni de completitud sobre la forma del árbol.

1.1 - Recursión estructural

Dar el tipo y definir una función `foldHeap`, que abstraiga el esquema de recursión estructural sobre heaps.

1.2 - Aplicación de funciones en los valores

Sin usar recursión explícita, dar el tipo y definir una función `mapHeap`, que aplique una función dada a todos los valores almacenados en el heap, sin modificar las prioridades ni la estructura.

1.3 - Recursión primitiva

Dar el tipo y definir una función `recHeap`, que abstraiga el esquema de recursión primitiva sobre heaps.

1.4 - Función prioridades

Sin usar recursión explícita, definir la función:

```
prioridades :: Heap a -> [Int]
```

que devuelva la lista de todas las prioridades almacenadas en el heap.

Por ejemplo:

```
prioridades h1
-- [1,3,5,8]
```

No importa el orden en que aparezcan las prioridades, siempre que estén todas y no se repitan artificialmente.

1.5 - Invariante del heap

Sin usar recursión explícita, definir la función:

```
esHeap :: Heap a -> Bool
```

que indique si el heap satisface el invariante de min-heap definido más arriba.

Se deberá usar alguno de los esquemas de recursión definidos en los incisos anteriores y justificar por qué resulta adecuado para esta función.

Ejercicio 2 - Cálculo lambda tipado

Se considera el cálculo lambda simplemente tipado visto en la materia, con valores booleanos, condicionales, funciones y las reglas usuales de tipado y evaluación *small-step*.

2.1 - Propiedades fundamentales

Explicar tres de las siguientes propiedades del cálculo lambda tipado, indicando en cada caso:

- qué afirma la propiedad;
- por qué es relevante para la evaluación de programas;
- qué tipo de error o comportamiento descarta.

Propiedades posibles:

1. Preservación de tipos.
2. Progreso.
3. Determinismo de la evaluación.
4. Unicidad de tipos.
5. Normalización en el cálculo lambda simplemente tipado.

No alcanza con nombrar las propiedades: se debe explicar su significado en términos de programas bien tipados y evaluación.

2.2 - Extensión de un cálculo

Se quiere extender el cálculo lambda tipado con una nueva construcción sintáctica. Explicar qué decisiones deben tomarse para que la extensión quede bien definida.

En particular, indicar qué debe definirse o revisarse en relación con:

1. la sintaxis de términos;
2. la sintaxis de tipos, si hiciera falta;
3. el conjunto de valores;
4. las reglas de tipado;
5. las reglas de evaluación;

Para cada punto, explicar brevemente qué podría salir mal si no se lo define de manera precisa.

2.3 - Una extensión no determinista

Se agrega al lenguaje una construcción $M \oplus N$, con la siguiente regla de tipado:

$$\frac{\Gamma \vdash M : \tau \quad \Gamma \vdash N : \tau}{\Gamma \vdash M \oplus N : \tau}$$

Además, se agregan las siguientes reglas de evaluación:

$$\frac{M \rightarrow M'}{M \oplus N \rightarrow M' \oplus N}$$

$$\frac{N \rightarrow N'}{V \oplus N \rightarrow V \oplus N'}$$

$$V_1 \oplus V_2 \rightarrow V_1$$

$$V_1 \oplus V_2 \rightarrow V_2$$

donde V , V_1 y V_2 representan valores.

Responder:

- a) Dar un ejemplo concreto de término cerrado y bien tipado cuya evaluación pueda terminar en dos valores distintos.
- b) Indicar cuál de las propiedades discutidas en el inciso 2.1 se pierde con esta extensión. Justificar.

- c) Indicar cuáles de las propiedades discutidas en el inciso 2.1 se conservan, al menos intuitivamente. Justificar.
- d) Explicar si la extensión introduce nuevos errores de tipado o si solamente cambia el comportamiento operacional del lenguaje.

2.4 - Recuperar determinismo

Proponer una modificación de las reglas de evaluación de $M \oplus N$ para que la evaluación vuelva a ser determinista.

La modificación debe conservar la regla de tipado anterior y debe seguir permitiendo evaluar términos de la forma $M \oplus N$.

Indicar claramente:

1. qué reglas se eliminan o reemplazan;
2. cuál sería el resultado de evaluar $true \oplus false$;
3. por qué con las nuevas reglas ya no hay dos próximos pasos posibles para un mismo término.

~~Indicar también un ejemplo de término que no sea un valor y que no pueda ser evaluado.~~

2.5 - Preservación para la extensión

Justificar informalmente que la extensión con $M \oplus N$ satisface preservación de tipos.

Es decir, justificar que si:

$$\Gamma \vdash M \oplus N : \tau$$

y:

$$M \oplus N \rightarrow P$$

entonces:

$$\Gamma \vdash P : \tau$$

La justificación debe considerar los distintos tipos de reglas de evaluación de $M \oplus N$:

- reducción del lado izquierdo;
- reducción del lado derecho;
- selección del primer valor;
- selección del segundo valor.

2.6 - Progreso para la extensión

Analizar informalmente si la extensión con $M \oplus N$ satisface progreso.

En particular, explicar qué ocurre cuando:

1. M todavía puede reducir;
2. M ya es un valor y N todavía puede reducir;
3. ambos lados ya son valores.

Concluir si todo término cerrado y bien tipado de la forma $M \oplus N$ es un valor o puede dar un paso de evaluación.

Ejercicio 3 — Programación lógica y Resolución

Se quiere modelar una red académica de relaciones entre investigadores usando Prolog. Para ello, se cuenta con hechos de la forma:

`publicaron(A, B).`

que indican que A y B publicaron conjuntamente. Esta relación debe interpretarse como simétrica: si A publicó con B, entonces B publicó con A.

También se cuenta con hechos de la forma:

`dirigio(Director, Dirigido).`

que indican que Director dirigió o codirigió la tesis doctoral de Dirigido. Para calcular distancias académicas, la relación de dirección se considerará bidireccional y tendrá distancia 0.5. Es decir, aunque dos investigadores no hayan publicado juntos, si uno dirigió la tesis doctoral del otro, se considera que están a distancia académica 0.5.

Las publicaciones conjuntas tendrán distancia 1.

Se provee la siguiente base de conocimientos:

```
publicaron(erdos, renyi).
publicaron(erdos, turan).
publicaron(erdos, bollobas).
publicaron(erdos, graham).
publicaron(bollobas, riordan).
publicaron(graham, knuth).
publicaron(graham, patashnik).
publicaron(knuth, patashnik).
```

```
dirigio(fejer, erdos).
dirigio(fejer, turan).
dirigio(erdos, bollobas).
dirigio(lehmer, graham).
dirigio(church, turing).
dirigio(hall, knuth).
```

Además, se proveen los siguientes predicados auxiliares:

```
% coautor(?A, ?B)
```

% A y B son coautores si publicaron conjuntamente. La relación se considera simétrica.

```
coautor(A, B) :- publicaron(A, B).
```

```
coautor(A, B) :- publicaron(B, A).
```

```
% vinculo_academico(?A, ?B, ?D)
```

% A y B tienen un vínculo académico directo de distancia D. Publicar conjuntamente tiene distancia 1. La c

```
vinculo_academico(A, B, 1) :- coautor(A, B).
```

```
vinculo_academico(A, B, 0.5) :- dirigio(A, B).
```

```
vinculo_academico(A, B, 0.5) :- dirigio(B, A).
```

A partir de esta base, resolver los siguientes incisos.

3.1)

Definir el predicado:

```
descendiente_academico(+C, -D)
```

que, dado un científico C, instancie en D a cada uno de sus descendientes académicos, directos o indirectos.

Diremos que D es descendiente académico directo de C si vale:

```
dirigio(C, D).
```

Además, si C dirigió a X y X tiene como descendiente académico a D, entonces D también es descendiente académico de C.

El predicado debe poder devolver, por backtracking, hijos, nietos, bisnietos académicos, etc.

Por ejemplo:

```
?- descendiente_academico(fejer, D).
```

```
D = erdos ;
```

```
D = turan ;
```

```
D = bollobas ;
```

```
...
```


3.2)

Definir el predicado:

`camino_academico(+A, +B, -Camino, -D)`

que, dados dos científicos A y B, genere caminos académicos simples entre ellos.

Un camino académico puede usar tanto publicaciones conjuntas como vínculos de dirección doctoral. La lista **Camino** debe comenzar en A, terminar en B y no debe repetir científicos.

La distancia total D debe ser la suma de las distancias de los vínculos utilizados, considerando:

- distancia 1 para publicaciones conjuntas;
- distancia 0.5 para vínculos de dirección doctoral.

Por ejemplo, una consulta posible es:

?- `camino_academico(renyi, bollobas, C, D)`.

Una respuesta posible es:

`C = [renyi, erdos, bollobas],`

`D = 1.5`

porque **renyi** publicó con **erdos**, y **erdos** dirigió a **bollobas**.

3.3)

Definir el predicado:

`distancia_academica(+A, +B, -D)`

que instancie D con la mínima distancia académica entre A y B.

Si hay varios caminos distintos con la misma distancia mínima, alcanza con devolver la distancia una sola vez.

Se debe considerar que:

`distancia_academica(A, A, 0)`.

para cualquier científico A registrado en la base de conocimientos.

Ejemplos esperados:

?- `distancia_academica(erdos, renyi, D)`.

`D = 1.`

?- `distancia_academica(erdos, bollobas, D)`.

`D = 0.5.`

?- `distancia_academica(renyi, bollobas, D)`.

`D = 1.5.`

3.4)

Analizar si el predicado `distancia_academica/3` definido en el inciso anterior es reversible en sus argumentos.

En particular, discutir qué ocurre con consultas como:

?- `distancia_academica(erdos, X, D)`.

y:

?- `distancia_academica(X, Y, D)`.

Justificar la respuesta distinguiendo entre:

- el significado declarativo del predicado;
- el comportamiento operacional esperable bajo la estrategia usual de Prolog.

3.5)

Considerar ahora la siguiente definición alternativa del predicado `relacionado/2`, que intenta indicar que dos científicos están conectados por algún camino académico:

```
relacionado(A, B) :-  
    relacionado(A, C),  
    vinculo_academico(C, B, _).
```

```
relacionado(A, B) :-  
    vinculo_academico(A, B, _).
```

Responder:

1. ¿La definición es declarativamente razonable?
2. ¿Qué ocurre en Prolog con la consulta siguiente?
`?- relacionado(erdos, renyi).`
3. Explicar por qué, bajo la estrategia usual de Prolog, la consulta puede no finalizar.
4. Mostrar cómo puede demostrarse `relacionado(erdos, renyi)` usando Resolución General.
5. Indicar cuál es la diferencia entre esta demostración y la ejecución de Prolog.

Examen Final

Paradigmas de Lenguajes de Programación
 Departamento de Computación, FCEyN, UBA
 25 de julio de 2025

1a	1b	1c	2	3	4a	4b	5a	5b	Nota
----	----	----	---	---	----	----	----	----	------

Incluir nombre, fecha y firma en todas las hojas.

Ejercicio 1 (Programación funcional)

Considerar la siguiente función:

```
foldu z f []      = z
foldu z f (x : xs) = f x (foldu z f xs)
```

- Dar el tipo de `foldu`.
- Definir `foldr` sin usar recursión explícita, usando `foldu`.
- Definir `foldu` sin usar recursión explícita, usando `foldr`.

Ejercicio 2 (Razonamiento ecuacional)

Notamos `Nat` al tipo de los naturales (enteros no negativos). Contamos con la operación que devuelve el máximo entre dos naturales:

```
max :: Nat -> Nat -> Nat
max x y = if x > y then x else y
```

Se asume ya demostrado que la operación `max` es conmutativa, asociativa y que 0 es el elemento neutro, es decir:

- Conmutatividad:** $\forall x y :: \text{Nat}. \text{max } x y = \text{max } y x.$
- Asociatividad:** $\forall x y z :: \text{Nat}. \text{max } x (\text{max } y z) = \text{max } (\text{max } x y) z.$
- Elemento neutro:** $\forall x :: \text{Nat}. \text{max } 0 x = x.$

Se definen además dos funciones `maxr` y `maxl` para calcular el máximo elemento de una lista de números naturales:

```
maxr :: [Nat] -> Nat
{MRO} maxr []      = 0
{MR1} maxr (x : xs) = max x
                    (maxr xs)

maxl :: Nat -> [Nat] -> Nat
{ML0} maxl ac []      = ac
{ML1} maxl ac (x : xs) =
    maxl (max x ac) xs
```

Las funciones `(++)` y `reverse` son las usuales:

```
(++) :: [a] -> [a] -> [a]
{A0} []      ++ ys = ys
{A1} (x : xs) ++ ys = x : (xs++ys)

reverse :: [a] -> [a]
{R0} reverse []      = []
{R1} reverse (x : xs) = reverse xs ++ [x]
```

Demostrar que $\forall xs :: [\text{Nat}]. \text{maxr } (\text{reverse } xs) = \text{maxl } 0 \text{ xs}.$

Ejercicio 3 (Programación lógica)

Un *árbol general* es o bien el símbolo x o bien una lista de árboles generales. Por ejemplo, los siguientes son árboles generales:

x $[]$ $[x, x]$ $[[x, x], x, [x, x]]$ $[x, [x, x, [x, [], [x, x]], x], [x, []]]$

Definir en Prolog un predicado `arbolGeneral(-A)` que genere todos los posibles árboles generales.

Ejercicio 4 (Deducción natural)

Se define un nuevo conectivo binario $\oplus$, con las siguientes reglas de introducción y eliminación:

$$\frac{\Gamma, \neg\tau, \neg\sigma \vdash \perp}{\Gamma \vdash \tau \oplus \sigma} \oplus_i \qquad \frac{\Gamma \vdash \tau \oplus \sigma \quad \Gamma, \tau \vdash \perp \quad \Gamma, \sigma \vdash \perp}{\Gamma \vdash \perp} \oplus_e$$

Dar derivaciones para los siguientes secuentes:

- a) $\tau \vee \sigma \vdash \tau \oplus \sigma$
- b) $\tau \oplus \sigma \vdash \tau \vee \sigma$

Ejercicio 5 (Resolución)

Dar ejemplos de fórmulas que producen el siguiente comportamiento en el método de resolución.

- a) Hay secuencias de aplicaciones de la regla de resolución que producen infinitas cláusulas distintas. **Hay** una refutación que conduce a la cláusula vacía.
- b) Hay secuencias de aplicaciones de la regla de resolución que producen infinitas cláusulas distintas. **No hay** una refutación que conduce a la cláusula vacía.

Examen Final

Paradigmas de Programación (plan 2023)
 Departamento de Computación, FCEyN, UBA
 1 de agosto de 2025

1	2	3	4	Nota
---	---	---	---	------

Incluir nombre, fecha y firma en todas las hojas.

Ejercicio 1 (Razonamiento ecuacional)

Consideramos el tipo de datos de los árboles binarios:

```
data AB a = Nil | Bin (AB a) a (AB a)
```

y las siguientes funciones:

```
{P0} post Nil          = []
{P1} post (Bin i x d)  = post d ++ post i ++ [x]
{M0} map f []          = []
{M1} map f (x : xs)    = f x : map f xs
{A0} mapA f Nil        = []
{A1} mapA f (Bin i x d) = [f x] ++ mapA f i ++ mapA f d
{R0} rev ac []         = ac
{R1} rev ac (x : xs)   = rev (x : ac) xs
```

Demostrar que para toda $f :: a \rightarrow b$ vale

$$\text{map } f \circ \text{post} = \text{rev } [] \circ \text{mapA } f.$$
Ejercicio 2 (Programación funcional / resolución)

Suponemos que contamos con los siguientes tipos de datos y funciones en Haskell para modelar la resolución SLD:

- **CláusulaDeDefinición** — tipo de las cláusulas de definición.
- **CláusulaObjetivo** — tipo de las cláusulas objetivo.
- **esVacía** $:: \text{CláusulaObjetivo} \rightarrow \text{Bool}$ — función que indica si una cláusula es $\square$.
- **resolvente** $:: \text{CláusulaObjetivo} \rightarrow \text{CláusulaDeDefinición} \rightarrow \text{CláusulaObjetivo}$ — función que calcula la resolvente entre dos cláusulas.

Definir una función

```
existeRefutaciónSLD :: [CláusulaDeDefinición] -> CláusulaObjetivo -> Bool
```

que indique si existe una refutación SLD. En caso de que la refutación exista, la función debe devolver **True**. En caso de que la refutación no exista, la función debe devolver **False**, o posiblemente indefinirse (por no terminación).

Ejercicio 3 (Deducción natural)

Extendemos las fórmulas de la lógica de primer orden con un nuevo cuantificador $\odot x.\tau$ con las siguientes reglas de introducción y eliminación:

$$\frac{\Gamma, \neg\tau\{x := t\} \vdash \perp}{\Gamma \vdash \odot x.\tau} \odot_i \qquad \frac{\Gamma \vdash \odot x.\tau \quad \Gamma \vdash \neg\tau \quad x \notin \text{fv}(\Gamma)}{\Gamma \vdash \perp} \odot_e$$

Demostrar que $\vdash (\odot x.P(f(x))) \Rightarrow \odot x.P(x)$.

Ayuda: el significado intuitivo del nuevo cuantificador es equivalente al del cuantificador existencial usual.

Ejercicio 4 (Programación lógica)

Representamos en Prolog las fórmulas de la lógica proposicional con los símbolos de función binarios **and** (conjunción), **or** (disyunción), **imp** (implicación), el símbolo de función unario **neg** (negación), y el símbolo de función unario **prop** de tal modo que **prop**(**n**) representa la n -ésima variable proposicional. Por ejemplo, la fórmula $P_1 \vee \neg(P_2 \wedge \neg P_2)$ se representa como:

```
or(prop(1), neg(and(prop(2), neg(prop(2)))))
```

Se puede asumir que en una fórmula **prop**(**n**) el valor de n siempre es positivo. Definir un predicado **esTautologia(+Form)** que, dada una fórmula **Form**, sea verdadero si todas las valuaciones la satisfacen.

Ejercicio 1

- Dos tipos son isomorfos si existen 2 funciones:
 - $f :: a \rightarrow b$,
 - $g :: b \rightarrow a$
 - tales que $f \cdot g = \text{id}$ y $g \cdot f = \text{id}$
- Teniendo los siguientes tipos de dato:

```
Either a b = left a | right b
AB a = nil | bin a (AB a) (AB a)
AX a b = vacio | nodoA a (AX a b) (AX a b) | nodoAB a b (AX a) (AX a)
```

- Demostrar que los tipos $(AX\ a\ b)$ y $AB\ (\text{either}\ a\ (a,b))$ son isomorfos.

Ejercicio 2

- Se cuenta con cálculo lambda extendido con booleanos y pares.
- Se quiere extender para introducir **pattern matching**.
- El patrón **P** puede tomar las siguientes formas:

```
P ::= x | V | <P, P>
```

- Que representan una variable, un valor, y un par de patrones respectivamente.
- Se agrega el siguiente término:

```
Match P ~ M1 then M2 else M3
```

- El comportamiento esperado es el siguiente:
 - Se evalua M1 hasta obtener un valor.
 - Si ese valor "concuerta" con P, se procede a evaluar M2.
 - De lo contrario se evalua M3.
- P y M1 "concuertan" de la siguiente manera:
 - Si P es una variable **x**, concuerda con cualquier valor ligandose a ese valor.
 - Si P es un valor, solo concuerda con el mismo valor.
 - Si P es un par $\langle P1, P2 \rangle$, concuerda con otro valor si ese valor es un par $\langle x, y \rangle$ donde P1 concuerda con x y P2 concuerda con y.
- a) Extender las reglas de tipado para que los patrones esten tipados correctamente.
- b) dar una derivación de tipo para la siguiente expresión:
 - $\text{Match } \langle x, y \rangle \sim p \text{ then } \langle y, x \rangle \text{ else } p$
- c) Extender el conjunto de valores y dar las nuevas reglas de semántica.
- d) reducir la siguiente expresión:

- Match $\langle x, y \rangle \sim \langle 1, 2 \rangle$ then $\langle y, x \rangle$ else $\langle 0, 0 \rangle$

Ejercicio 3

- Dos listas l_1 y l_2 son componibles si tienen un elemento en común que los permite "concatenar"
 - Por ejemplo $[1, 2, 3]$ es componible con $[3, 4, 5]$ y la concatenación de esas dos listas es $[1, 2, 3, 4, 5]$.
- Se extiende de manera obvia a n listas, $[l_1, \dots, l_n]$:
 - Por ejemplo $[1, 2] [2, 3] [3, 4] [4, 5]$ son componibles y resulta en la lista $[1, 2, 3, 4, 5]$
- Un rompecabezas es una lista de listas tal que son componibles para algún orden:
 - Ej la solución $[1, 2, 3, 4, 5]$ tiene como rompecabezas la lista $[[2, 3, 4], [1, 2], [4, 5]]$ (en este caso es la única solución)
- Implementar el predicado generarRompecabezas(+S, -R) que dado una lista solución S , instancia en R todos los rompecabezas que tienen a S como solución.

Ejercicio 4

- Se extiende deducción natural con las siguientes reglas:

$$\frac{\Gamma \vdash \alpha \quad \Gamma, \tau \vdash \perp}{\Gamma \vdash \alpha \Theta \tau} \text{ } \theta i$$

$$\frac{\Gamma \vdash \alpha \Theta \tau \quad \Gamma, \alpha \vdash \tau}{\Gamma \vdash \rho} \text{ } \theta e$$

- Demostrar que:
 - a) $\neg(\alpha \Rightarrow \tau) \vdash \alpha \Theta \tau$
 - b) $\neg(\alpha \Theta \tau) \vdash \alpha \Rightarrow \tau$

Final 17/12/25 (PLP)

Sumario

- 1 Ejercicio 1
- 2 Ejercicio 2
- 3 Ejercicio 3
- 4 Ejercicio 4

Ejercicio 1

Programación Funcional

Sean los siguientes tipos:

```
data Indice = I1 | I2 | I3
data AT a = Hoja a | Nodo (Indice → AT a)
```

En donde AT a representa el tipo de dato "Árbol Trébol", dar:

- a. Dar el tipo e implementar foldAT
- b. Definir la función altura :: AT a → Int
- c. Dar el tipo e implementar mapAT que, dada una función, se la aplica a todas las hojas de un árbol trébol.

Todo nodo SIEMPRE tiene 3 árboles trébol asociados.

Ejercicio 2

Cálculo Lambda

Se extiende el Cálculo Lambda con el operador Δ . Funciona de la siguiente manera:

Sean M, N términos, entonces $M\Delta N$ reduce M y N de manera intercalada. Primero reduce en un paso a M, luego reduce en un paso a N, luego reduce en otro paso a M, y así sucesivamente.

Cuando M o N lleguen a un valor, entonces ese valor es el resultado de reducir $M\Delta N$.

- a. Extender los términos y reglas de tipado.
- b. Extender los valores y las reglas semánticas. Decidir qué hacer en el caso de que M y N sean valores.
- c. Dar 3 términos cerrados F, M1 y M2 tal que:

```
F es de tipo Bool → Bool
M1 es de tipo Bool
M2 es de tipo Bool
```


y se cumpla que:

```
M1  $\leftrightarrow$  true  
M2  $\leftrightarrow$  true  
F M1  $\leftrightarrow$  true  
F M2  $\leftrightarrow$  false
```

(Las flechas dobles representan "muchos pasos").

Ejercicio 3

Programación Lógica

Este era un ejercicio que tomó Pablo, sobre armar "rompecabezas". Si lo buscan en algún parcial viejo lo encuentran.

El predicado que pedía armar era:

`generarRompecabezas(+S, -R)`

Ejercicio 4

Unificación

Verdadero y falso. Demostrar en ambos casos:

Sea G un conjunto de goals, y $S1$ y $S2$ unificaciones. Decimos que $S1 \leq S2$ si $S1$ es más general que $S2$.

- a. Vale siempre que $S1 \leq S2$ o que $S2 \leq S1$.
- b. Si $S1 \leq S2$ y $S2 \leq S1$, entonces $S1 = S2$.
- c. Siempre sucede que existe T tal que $T \leq S1$ y $T \leq S2$ (independientemente del G).

Obtenido de «[https://www.cubawiki.com.ar/index.php?title=Final_17/12/25_\(PLP\)&oldid=19455](https://www.cubawiki.com.ar/index.php?title=Final_17/12/25_(PLP)&oldid=19455)»

Nombre y apellido: LU: DNI:
E-mail: # Hojas: Firma:

Poner nombre, fecha y firma en cada hoja. Justificar todas las respuestas.

1.a	1.b	1.c	1.d	1.e	2.a	2.b	2.c	3.a	3.b	3.c	Nota

Ejercicio 1 (Programación funcional). El tipo algebraico `ABBComp` describe a los *árboles binarios de búsqueda (ABB) parcialmente comprimidos* con elementos de tipo `a` y sin repetidos. Los `ABBComp` son árboles que pueden tener algún subárbol donde, en vez de seguir con una estructura arbórea, sus elementos se incluyen directamente en una lista ordenada (se asume que `a` pertenece a la clase `Ord`).

`data ABBComp a = Nil | Comp [a] | Nodo a (ABBComp a) (ABBComp a)`

Por ejemplo, las siguientes expresiones denotan árboles de tipo `ABBComp Int` que denotan los mismos elementos:

- `Comp [0,2,14,22]`
- `Nodo 2 (Nodo 0 Nil Nil) (Comp [14,22])`
- `Nodo 14 (Nodo 2 (Nodo 0 Nil Nil) Nil) (Nodo 22 Nil Nil)`

- a) Dar el tipo y definir una función `foldABBComp`, que abstraiga el esquema de recursión estructural sobre los *ABB parcialmente comprimidos*.
- b) Sin usar recursión explícita, dar el tipo y definir una función `mapABBComp` que aplique una función dada a todos los valores de un árbol del tipo indicado anteriormente.
- c) Dar el tipo y definir `recABBComp` que abstraiga el esquema de recursión primitiva sobre estos árboles.
- d) Sin usar recursión explícita, dar el tipo y definir la función `ordenado`, que permitirá verificar el invariante de la estructura (i.e., que los elementos de un `ABBComp` estén ordenados).
- e) Dar el tipo y definir la función `iguales(a1,a2)`, que recibe dos `ABBComp` y devuelve `True` si son iguales al recorrerlos *inorder*. Se deberá usar alguno de los esquemas de recursión anteriores y justificar por qué se lo usó.

Ejercicio 2 (Cálculo-λ). Se extiende el cálculo-λ para permitir una *maratón de términos* del mismo tipo, a partir del agregado de la extensión ganador $[M_1, \dots, M_n]$ (con $n \geq 1$). Así, el resultado de su evaluación será la del subtérmino cuya evaluación finaliza en primer lugar, yendo en rondas donde se recorren los subtérminos de izquierda a derecha (i.e., en cada ronda se evalúa un paso de cada subtérmino, si ya no es un valor, hasta recorrer todos ellos).

Por ejemplo, ganador $[if (\lambda x : bool. x) false then true else false, true, (\lambda x : bool. x) false] \rightarrow true$.

- a) Extender formalmente la gramática de los términos y el sistema de tipos.
- b) Extender la gramática de los valores, en caso de ser necesario, y las reglas de semántica operacional *small-step*. Tomar una decisión sobre lo que sucede cuando hay varios subtérminos que finalizan en la misma ronda.
De ser necesario, se pueden considerar definiciones adicionales para poder tener control de en qué paso de la evaluación se encuentra cada subtérmino. Justificar adecuadamente.
- c) Explicar detalladamente qué se modificaría en los puntos anteriores para la extensión perdedor $[M_1, \dots, M_n]$, que es igual a la anterior, pero donde el resultado de la evaluación es la del último subtérmino que finaliza su evaluación.

Ejercicio 3 (Programación Lógica). A las listas comunes de números enteros (e.g., $[8,1,1,1, 10,10,2]$) se le quiere sumar un nuevo tipo de elemento que son los comprimidos (anotados como $c(N,K)$, donde N es el número y K es la cantidad de repeticiones, con $K \geq 1$). Así, podremos generar listas *comprimidas*, donde se puedan agrupar los elementos contiguos repetidos, indicando la cantidad de valores iguales por medio un par de valores (obs.: si se comprime ciertos elementos contiguos, se deben incluir todos ellos, por lo que, por ejemplo, $[8,c(1,2), 1,10,10,2]$ no sería una compresión válida para el caso anterior). No es obligatorio comprimir toda la lista para que esta sea *comprimida*. Sólo por nombrar algunas, la lista anterior podría tener distintas versiones comprimidas: $[c(8,1), c(1,3), 10,10,2]$, $[8,c(1,3), c(10,2), 2]$, $[c(8,1), c(1,3), c(10,2), c(2,1)]$, y $[8,1,1,1,10,10,2]$.

- a) Definir un predicado `comprimido(+L1, -L2)` que dada una lista `L1`, `L2` es una versión comprimida de `L1`. No deben generarse soluciones `L2` idénticas más de una vez.
- b) ¿El predicado `comprimido` puede ser reversible en uno o ambos argumentos? Justificar detalladamente.
- c) Definir un predicado `iguales(+L1, +L2)`, que indique si dos listas son observacionalmente iguales, considerando que ambas denotan la misma lista de números enteros. Se pueden agregar predicados auxiliares.

Nombre y apellido: LU: DNI:
E-mail: # Hojas: Firma:

Poner nombre, fecha y firma en cada hoja. Justificar todas las respuestas.

1.a	1.b	1.c	1.d	2.a	2.b	2.c	2.d	3.a	3.b	3.c	3.d	Nota

Ejercicio 1 (Cálculo- λ). Sean M, N dos programas (es decir, términos cerrados y tipables) que además tienen el mismo tipo. Decimos que M es *distinguible* de N si existe una lista finita de valores $V_1, \dots, V_n$, con $n \geq 0$, tales que $M V_1 \dots V_n \rightarrow^* V_a$ y $N V_1 \dots V_n \rightarrow^* V_b$, con $V_a \neq V_b$.

- Demostrar que el programa $\lambda f : \text{Bool} \rightarrow \text{Bool}. f (f \text{ true})$ es distinguishible del programa $\lambda f : \text{Bool} \rightarrow \text{Bool}. f \text{ true}$.
- Decimos que M es indistinguishible de N si no vale que M es distinguishible de N . Es fácil de ver que M es indistinguishible de M . Exhibir dos programas M y N *distintos*, tales que M sea indistinguishible de N .
- Decidir si la siguiente afirmación es verdadera o falsa y justificar: si M es distinguishible de N , entonces N es distinguishible de M .
- Decidir si la siguiente afirmación es verdadera o falsa y justificar:
si M es distinguishible de N , con $M, N : T$, entonces $\lambda f : T \rightarrow \text{Bool}. f M$ es distinguishible de $\lambda f : T \rightarrow \text{Bool}. f N$.

Ejercicio 2 (Programación Lógica y Resolución).

Se cuenta con la siguiente base de conocimientos sobre relaciones familiares:

```
progenitor(diego,dalma).      progenitor(chitoro,ana).      pareja(chitoro,tota).
progenitor(diego,gianinna).  progenitor(ana,daniel).      pareja(diego,claudia).
progenitor(tota,diego).      pareja(gianinna,osvaldo).    pareja(ana,pedro).
```

```
pareja(X,Y) :- pareja(Y,X).
ancestro(A, X) :- progenitor(A, X).
ancestro(A, X) :- progenitor(A, Y), ancestro(Y, X).
```

Donde $\text{progenitor}(P, H)$ indica que P es progenitor de H , $\text{pareja}(X, Y)$ indica que X e Y están en pareja, y $\text{ancestro}(A, X)$ indica que A es un ancestro (directo o indirecto) de X .

- Definir un predicado $\text{descendientes}(+P, -L)$, que instancie en L la lista de todos los descendientes de P , sin repetidos. Obs.: No se exige ningún orden particular en la lista resultante.
- Analizar y justificar detalladamente si uno o ambos argumentos del predicado anterior pueden ser reversibles.
- Definir un predicado $\text{ancestroComunMasCercano}(+P1, +P2, -A)$, que se satisfaga cuando A es el ancestro común más cercano entre $P1$ y $P2$. Si hay más de uno, debería dar todos ellos.
Por ejemplo, si dos personas son hermanas, sus ancestros comunes más cercanos serían sus progenitores.
- Comparar qué pasa en Prolog y en Resolución General ante una fórmula como la de abajo, y explicar en detalle por qué podrían haber diferencias:

$\exists A. (\text{ancestro}(A, \text{daniel}) \wedge \text{ancestro}(A, \text{gianinna}))$

(El examen sigue al dorso $\rightarrow$)

Ejercicio 3 (Programación funcional). El tipo algebraico `AdjAB a` describe a las *adjunciones de árboles binarios* con elementos de tipo `a`. Las `AdjAB` representan listas de árboles binarios, donde cada uno de ellos está al mismo nivel que el otro, que pueden o no tener una raíz en común de acuerdo al constructor usado.

```
data AdjAB a = Raiz a [AB a] | Adj [AB a]
data AB a = Nil | Hoja a | Arb a (AB a) (AB a)
```

Por ejemplo, las siguientes expresiones denotan distintos árboles de tipo `AdjAB Int`:

- `Adj [Nil, Nil, Arb 3 Nil Nil, Hoja 6]`
- `Raiz 5 [Arb 3 (Arb 1 Nil Nil) (Hoja 4), Arb 3 Nil Nil, Nil]`
- `Raiz 10 []`

- a) Dar el tipo y definir una función `foldAdjAB`, que abstraiga el esquema de recursión estructural sobre las *adjunciones de árboles binarios*, recorriendo todos los árboles por completo.
- b) Sin usar recursión explícita, dar el tipo y definir una función `mapAdjAB` que aplique una función dada a todos los valores de una adjunción.
- c) Dar el tipo y definir `recAdjAB` que abstraiga el esquema de recursión primitiva sobre las adjunciones, recorriendo todos los árboles por completo.
- d) Sin usar recursión explícita, dar el tipo y definir la función `ordenado`, que permita saber si toda la adjunción está ordenada. Así, cada árbol debe estar ordenado como un ABB y, adicionalmente, si hubiera raíz común, esta debe ser mayor que la raíz de todos los árboles. En el caso de `Nil`, este árbol está trivialmente ordenado y, además, se lo puede ignorar para comparar con la raíz común.

Nombre y apellido: LU: DNI:
E-mail: # Hojas: Firma:

Poner nombre, fecha y firma en cada hoja. Justificar todas las respuestas.

1.a	1.b	1.c	1.d	1.e	2.a	2.b	2.c	2.d	3.a	3.b	3.c	Nota

Ejercicio 1 (Programación funcional). El tipo algebraico `Trie a` describe los árboles de tipo *trie* (del inglés *reTRIEval*), que se puede usar en forma *naive* para representar conjuntos o diccionarios.

```
data Trie a = TrieNode (Maybe a) [(Char, Trie a)]
```

El camino desde la raíz hasta un nodo determina una cadena. Si el nodo contiene un valor de la forma `Just v`, entonces esa cadena representa una palabra del conjunto o una clave definida del diccionario. Si contiene `Nothing`, entonces la cadena no está definida como palabra o clave, aunque puede ser prefijo de otras cadenas.

Decimos que la representación es *naive*, porque la lista de hijos puede contener más de una aparición del mismo carácter. Por eso, una misma cadena puede estar representada por más de un camino dentro del *trie*.

Por ejemplo:

```
t :: Trie Bool
t =
  TrieNode Nothing
    [ ('a', TrieNode (Just True) [])
    , ('b', TrieNode Nothing
      [ ('a', TrieNode (Just True)
        [ ('d', TrieNode Nothing []) ])
      ])
    , ('c', TrieNode (Just True) [])
    ]
```

- Dar el tipo y definir una función `foldTrie`, que abstraiga el esquema de recursión estructural sobre los *tries*.
- Sin usar recursión explícita, definir la función `palabras :: Trie a -> [[Char]]` que, dado un *trie*, devuelva todas las palabras incluidas en él. Una palabra está incluida si y solo si el nodo alcanzado por ese camino desde la raíz contiene un valor de la forma `Just _`, y esto incluye a la raíz, lo que implica que se incluye a la palabra vacía. No se deben incluir palabras repetidas. Por ejemplo, dado el *trie* anterior:
`palabras t == ["a", "ba", "c"]`
- Sin usar recursión explícita, dar el tipo y definir una función `mapTrie` que aplique una función dada a todos los valores de un *trie*.
- Se quiere definir una función `conLongitud :: Trie a -> Trie (a, Int)` que asocie a cada valor almacenado la longitud de la palabra correspondiente. ¿Puede definirse usando únicamente `mapTrie`? Justificar. En caso negativo, definir una función más general que permita implementarla.

Ejercicio 2 (Programación Lógica y Resolución).

Una *estructura arbórea* es o bien el símbolo `o` o bien una lista de estructuras arbóreas. Por ejemplo, las siguientes son estructuras arbóreas válidas:

`o` `[]` `[o,o]` `[[o,o],o,[o,o]]` `[o,[o,o,[o,[],[o,o]],o],[o,[]]]`

- Definir en Prolog un predicado `estructuraArborea(-E)` que genere todas las posibles estructuras arbóreas.
- Pasar la base de conocimientos a formal clausal.
- Dar un ejemplo de consulta que, bajo la estrategia usual de Prolog, no finalice en alguna situación.
- Mostrar que, usando Resolución General, puede responderse la consulta anterior. Explicar la diferencia con Prolog.

(El examen sigue al dorso →)

Ejercicio 3 (Cálculo- λ). Se extiende el cálculo- λ con un constructor de términos $M ::= \dots \mid M \bowtie N$. No se extienden los conjuntos de tipos ni el conjunto de valores. Se agregan las siguientes reglas de tipado y reducción:

$$\frac{\Gamma \vdash M : \tau \quad \Gamma \vdash N : \tau}{\Gamma \vdash M \bowtie N : \tau} \text{T-}\bowtie \quad \frac{M \rightarrow M'}{M \bowtie N \rightarrow M' \bowtie N} \text{E-}\bowtie_1 \quad \frac{M \rightarrow M'}{V \bowtie M \rightarrow V \bowtie M'} \text{E-}\bowtie_2$$

$$\frac{}{V_1 \bowtie V_2 \rightarrow V_1} \text{E-SELECT}_1 \quad \frac{}{V_1 \bowtie V_2 \rightarrow V_2} \text{E-SELECT}_2$$

Observar que se pierde el determinismo, ya que por ejemplo $\text{true} \bowtie \text{false} \rightarrow \text{true}$ y $\text{true} \bowtie \text{false} \rightarrow \text{false}$.

- Decidir si es verdadera o falsa y justificar: existe un valor V tal que $V \text{ true} \rightarrow^* \text{true}$ y también $V \text{ true} \rightarrow^* \text{false}$.
- Si M es un término cerrado y tipable, definimos $\mathcal{E}(M)$ como el conjunto de los valores V tales que $M \rightarrow^* V$. Por ejemplo, $\mathcal{E}(\text{true} \bowtie (\text{true} \bowtie \text{false})) = \{\text{true}, \text{false}\}$. Suponiendo que $\mathcal{E}(M)$ tiene un elemento y $M : \text{bool}$, definir M' tal que $\mathcal{E}(M')$ tenga dos elementos.
- Decidir si es verdadera o falsa y justificar: existe un término cerrado y tipable M tal que $\mathcal{E}(M (M (\text{true} \bowtie \text{false})))$ es un conjunto de 8 elementos.

Nombre y apellido:LU:

E-mail:# Hojas:Firma:

1a	1b	2	3a	3b	3c	4a	4b	4c	Nota

Incluir nombre, fecha y firma en todas las hojas.

Ejercicio 1 (Razonamiento ecuacional). Considerar las siguientes definiciones en Haskell:

{N0} nand1 b [] = b

{N1} nand1 b (x:xs) = nand1 (not x || b) xs

{A0} any f [] = False

{A1} any f (x:xs) = f x || any f xs

Demostrar la siguiente propiedad: nand1 False = any not

Se pueden utilizar las propiedades conocidas de || y not mencionándolas al momento de usarlas.

Ejercicio 2 (Deducción natural). Extendemos el sistema de deducción natural con un nuevo conectivo lógico binario Δ , que tiene las siguientes reglas de introducción y eliminación:

$$\frac{\Gamma, \tau, \sigma \vdash \perp}{\Gamma \vdash \tau \Delta \sigma} \Delta_i \quad \frac{\Gamma \vdash \tau \Delta \sigma \quad \Gamma, \neg \tau \vdash \rho \quad \Gamma, \neg \sigma \vdash \rho}{\Gamma \vdash \rho} \Delta_e$$

a) Demostrar que vale $\vdash \neg(\tau \wedge \sigma) \Rightarrow \tau \Delta \sigma$ para todo par de fórmulas τ y σ .b) Demostrar que vale $\tau \Delta \sigma \vdash \neg(\tau \wedge \sigma)$ para todo par de fórmulas τ y σ .**Ejercicio 3 (Cálculo Lambda).** Dos programas M y N de un mismo tipo son *distinguishables* si existen valores $V_0, V_1, \dots, V_n$ con $n \geq 0$ tales que $V_0(MV_1 \dots V_n) \rightarrow \text{True}$ y $V_0(NV_1 \dots V_n) \rightarrow \text{False}$ (o viceversa, lo cual es equivalente), e *indistinguishables* en caso contrario.

Indicar si cada una de las siguientes afirmaciones es verdadera o falsa y justificar:

a) Si existe M' tal que $M \rightarrow M'$ y $N \rightarrow M'$, entonces M y N son indistinguishables.b) Si existen V y V' tales que $MV \rightarrow V'$ y $NV \rightarrow V'$, entonces M y N son indistinguishables.c) Si $\vdash M : \tau \rightarrow \sigma$ es derivable, entonces M y $\lambda x : \tau. Mx$ son indistinguishables.**Ejercicio 4 (Programación lógica).** Se cuenta con el siguiente predicado para verificar que una fórmula en Lógica Proposicional esté bien formada:

```
%formula(+F)
formula(X) :- atom(X).
formula(not(P)) :- formula(P).
```

```
formula(and(P,Q)) :- formula(P), formula(Q).
formula(or(P,Q)) :- formula(P), formula(Q).
```

a) Definir el predicado `formula(+XS, -F)` que, dada una lista de átomos XS sin elementos repetidos, vaya instanciando en F todas las fórmulas cuyas variables proposicionales pertenecen a XS .

b) El predicado anterior, ¿es reversible en su segundo parámetro? Justificar explicando qué sucede si se pasa la fórmula instanciada.

c) Se dispone de un predicado `valuation(+F, -V)` que va instanciando en V todas las valuaciones que asignan valores de verdad a las variables de la fórmula F , y un predicado `eval(+V, +F)` que nos da el resultado de evaluar a F en la valuación V (la representación de las valuaciones no es importante).Definir el predicado `tautology(+F)`, que tenga éxito si F es una tautología (tener en cuenta que para que algo sea una tautología debe ser una fórmula).